

金融数据处理与收益率分析：从原始行情到可分析样本

肖亮海

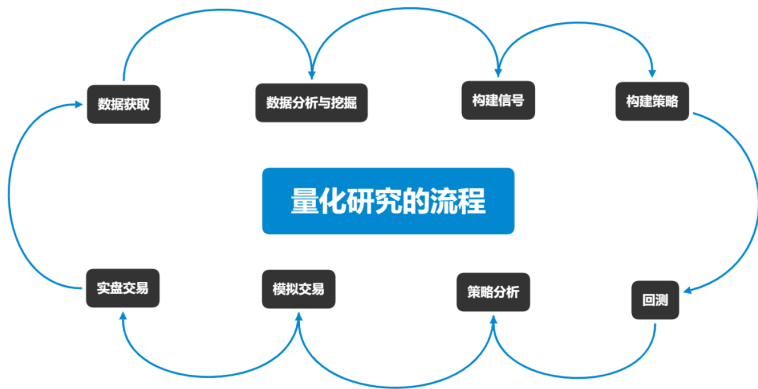
暨南大学 信息科学技术学院

课程 GitHub: github.com/styluck/quant

本讲位置

- 1 金融数据处理的基本流程
- 2 金融数据及其结构
- 3 价格口径、收益率与基础绩效
- 4 数据清洗与时间对齐
- 5 课堂实践

数据处理是量化研究的起点

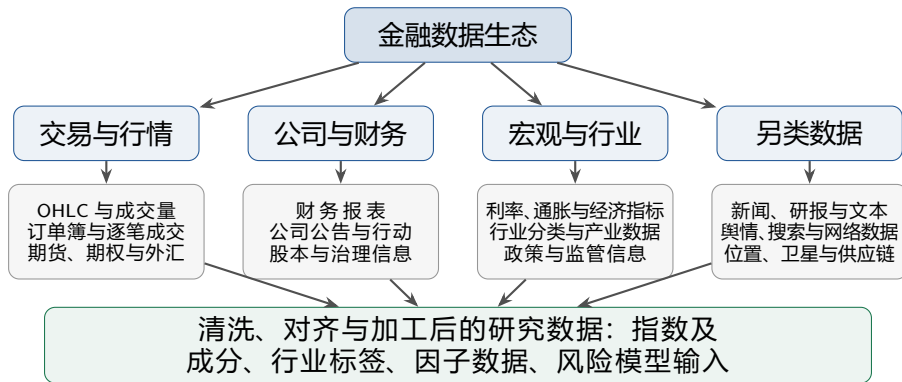


- 数据处理连接原始市场记录与后续因子、模型、组合和回测。
- 如果数据处理错误，后续模型越复杂，错误可能被放得越大。

本讲位置

- 1 金融数据处理的基本流程
- 2 金融数据及其结构**
- 3 价格口径、收益率与基础绩效
- 4 数据清洗与时间对齐
- 5 课堂实践

金融数据从哪里来?



原始数据来自不同渠道；进入模型前，必须统一字段、时间、样本和可得信息口径。

量化研究中常见的数据表

层级	常见数据表	典型字段	主要用途
原始	行情与交易	OHLC、成交量、成交额、订单簿、交易状态	收益率、流动性、执行分析
原始 原始	公司与财务 宏观、行业与另类	三大报表、公告、股本、分红与公司行动 利率、经济指标、产业数据、新闻与文本	价值、质量、成长、价格复权 资产配置、周期、事件与文本信号
加工 加工 加工	指数与基准 分类与状态标签 因子与风险数据	历史成分、权重、指数收益、全收益指数 行业、上市日期、ST、停牌、可交易状态 因子值、暴露、因子收益、协方差矩阵	股票池、市场基准、主动收益 样本筛选、中性化、交易约束 选股、风险归因、组合优化

关键区别

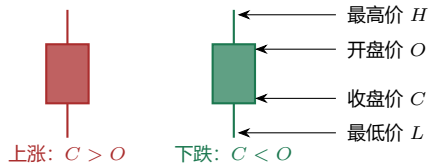
因子、指数和风险数据通常是加工结果，其可靠性取决于底层数据口径、历史样本和可得时间。

行情数据：字段、K 线与数据表

常见字段

- date, code: 日期、BaoStock 股票代码
- open: 开盘价
- high: 最高价
- low: 最低价
- close: 收盘价
- volume: 成交量
- amount: 成交额
- tradestatus: 交易状态

K 线：用 OHLC 描述一个时间区间



2025 年 1-6 月沪深 300 月线 (BaoStock)

月	Open	High	Low	Close	Volume (亿股)	Amount (亿元)
01	3931.8	3934.2	3704.1	3817.1	2741.1	52943.8
02	3844.7	3995.9	3785.2	3890.0	3399.6	71139.3
03	3899.7	4025.3	3855.7	3887.3	3264.1	62258.2
04	3892.8	3905.3	3514.1	3770.6	3328.4	57717.4
05	3794.4	3960.5	3782.8	3840.2	2351.8	43438.2
06	3833.5	3969.3	3827.1	3936.1	2838.3	52196.1

长表与宽表

长表：适合存储和清洗

date	code	close
2024-01-02	sh.600000	10.2
2024-01-02	sh.600004	20.5
2024-01-03	sh.600000	10.4
2024-01-03	sh.600004	20.1

宽表：适合矩阵计算

date	sh.600000	sh.600004
2024-01-02	10.2	20.5
2024-01-03	10.4	20.1

- 实践中常用长表读入和清洗，再用 pivot 转成宽表计算收益率矩阵。
- 本讲统一沿用 BaoStock 代码格式，例如浦发银行记为 sh.600000。

上节课作业参考答案（一）：准备股票列表

```
from pathlib import Path
import baostock as bs
import pandas as pd

def to_frame(rs): # 将 BaoStock 查询结果转为 DataFrame
    rows = []
    while rs.next():
        rows.append(rs.get_row_data())
    return pd.DataFrame(rows, columns=rs.fields)

bs.login()
basic = to_frame(bs.query_stock_basic()) # 获取证券基本信息
mask = (basic["type"] == "1") & basic["code"].str.startswith("sh.6")
codes = basic.loc[mask, "code"] # 保留上证 A 股代码
fields = ("date,code,open,high,low,close,preclose,volume,amount,"
          "turn,tradestatus,pctChg,isST")
field_list = fields.split(",") # CSV 表头字段
out = Path("data/raw/sse_daily")
out.mkdir(parents=True, exist_ok=True)
```

上节课作业参考答案（二）：按股票保存长表

```
for code in codes:
    rs = bs.query_history_k_data_plus(
        code, fields, start_date="2015-01-01", end_date="2025-12-31",
        frequency="d", adjustflag="3") # 下载不复权日线
    stock = to_frame(rs).reindex(columns=field_list)
    stock.to_csv(out / f"{code}.csv", header=True, index=False) # 长表

basic.to_csv("data/raw/sse_stock_basic.csv", index=False) # 保留上市信息
bs.logout()
```

上节课作业参考答案（三）：读取并生成宽表

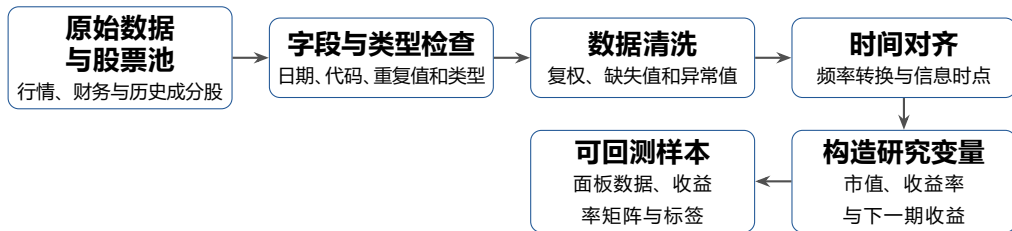
每个 CSV 保存一只股票的时间序列；现在从磁盘重新读取全部文件并合并：

```
files = sorted(out.glob("sh.6*.csv")) # 读取目录下全部股票文件
panel = pd.concat((pd.read_csv(f) for f in files), ignore_index=True)

num = ["open", "close", "volume", "amount"]
panel[num] = panel[num].apply(pd.to_numeric, errors="coerce")
wide = panel.pivot(
    index="date", columns="code", values=num
) # 行为日期，列为“字段--股票代码”
```

- panel 是合并后的长表；wide 是具有两层列索引的宽表。
- 例如 wide["close"] 是“日期 × 股票”的收盘价矩阵。

本讲研究主线：从原始数据到可回测样本



贯穿流程的质量检查

历史股票池是否完整？缺失值来自停牌还是数据错误？变量在组合形成时是否已经可得？样本中的股票和价格是否真实可交易？

数据处理是在决定策略使用什么信息、哪些股票以及什么价格，而不只是回测前的机械准备。

本讲位置

- 1 金融数据处理的基本流程
- 2 金融数据及其结构
- 3 价格口径、收益率与基础绩效**
- 4 数据清洗与时间对齐
- 5 课堂实践

价格口径：原始价格、复权价与全收益

- **原始价格**：当时真实交易价格，不调整分红、送股与拆股。
- **前复权价**：保持近期价格口径，向前调整历史价格。
- **后复权价**：保持早期价格口径，向后调整后续价格。
- **全收益口径**：假设现金分红再投资后的总回报。

研究收益率通常使用复权价或全收益口径；模拟真实下单则需要原始价格。



前复权与后复权走势相同，但锚定基准和价格尺度不同；后复权价使用右轴。

BaoStock: 查询复权因子

复权因子需要通过独立接口查询:

```
rs = bs.query_adjust_factor(
    code="sh.600000",
    start_date="2015-01-01",
    end_date="2025-12-31"
) # 查询浦发银行复权因子
assert rs.error_code == "0", rs.error_msg

rows = []
while rs.next():
    rows.append(rs.get_row_data())

factor = pd.DataFrame(
    rows, columns=rs.fields
) # 转为 DataFrame
print(factor.head())
```

关键返回字段

- code: 股票代码
- dividOperateDate: 除权除息日期
- foreAdjustFactor: 前复权因子
- backAdjustFactor: 后复权因子

复权因子不是
query_history_k_data_plus() 的
fields 字段。

如果只需要接口直接返回复权价格

adjustflag="1": 后复权; adjustflag="2": 前复权; adjustflag="3": 不复权。

为什么不能直接用原始收盘价

公司行为会改变每股价格或股数，但这种变化不一定代表投资者财富发生了同等幅度的变化。

1. 现金分红：价格下降，但收到现金

除息日前价格为 P ，每股分红为 D ，除息参考价约为 $P - D$ 。股东少了一部分股票市值，但同时收到现金 D 。

$$R^{\text{total}} \approx \frac{P_t + D}{P_{t-1}} - 1.$$

2. 股票拆分：股数增加，单价同比下降

一股拆成 k 股后，持股数由 N 变为 kN ，每股价格约由 P 变为 P/k ，总持仓价值基本不变。

$$NP = (kN) \frac{P}{k}.$$

3. 送股：免费获得股份，价格需要除权

每 10 股送 m 股后，持股数扩大为原来的 $1 + m/10$ 倍，除权参考价约为

$$P^{\text{ex}} = \frac{P}{1 + m/10}.$$

股数增加抵消了每股价格下降。

4. 配股：以配股价投入新资金

每股可按价格 S 认购 q 股时，理论除权价约为

$$P^{\text{ex}} = \frac{P + qS}{1 + q}.$$

价格变化同时涉及新增股份和认购现金，不能直接解释为投资损益。

如果直接使用原始收盘价：机械跳变会被误认为真实涨跌，进而污染收益率、动量、波动率和回撤，甚至制造虚假的交易信号。

定义

若资产价格为 P_t , 简单收益率定义为

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1.$$

- 简单收益率最直观, 组合收益也通常用简单收益率加权:

$$R_{p,t} = \sum_{i=1}^N w_{i,t-1} R_{i,t}.$$

- 注意: 收益率的价格 P_t 应该采用与研究问题一致的价格口径。

定义

对数收益率定义为

$$r_t = \ln \frac{P_t}{P_{t-1}} = \ln(1 + R_t).$$

- 对数收益率在时间上可加：

$$\sum_{t=1}^T r_t = \ln \frac{P_T}{P_0}.$$

- 当收益率很小时, $r_t \approx R_t$ 。
- 对数收益率常用于统计建模, 但组合收益不能简单用对数收益率线性加权。

小练习：在股票长表中加入后复权因子

修改上节课作业中的下载循环：查询每只股票的后复权因子，将其匹配到每个交易日，新增 `backAdjustFactor` 字段后再保存 CSV。

提示：复权因子只在除权除息日更新，其他交易日应沿用最近一次已经生效的因子。

小练习：在股票长表中加入后复权因子

修改上节课作业中的下载循环：查询每只股票的后复权因子，将其匹配到每个交易日，新增 `backAdjustFactor` 字段后再保存 CSV。

提示：复权因子只在除权除息日更新，其他交易日应沿用最近一次已经生效的因子。

Python 参考答案（放在 `for code in codes` 循环内）

```
frs = bs.query_adjust_factor(
    code, start_date="1990-01-01", end_date="2025-12-31"
)
factor = to_frame(frs)[["dividOperateDate", "backAdjustFactor"]]
factor = factor.rename(columns={"dividOperateDate": "date"})

stock["date"] = pd.to_datetime(stock["date"])
factor["date"] = pd.to_datetime(factor["date"])
stock = pd.merge_asof(
    stock.sort_values("date"), factor.sort_values("date"),
    on="date", direction="backward"
) # 匹配当日已经生效的最近因子
stock["backAdjustFactor"] = pd.to_numeric(
    stock["backAdjustFactor"], errors="coerce"
).fillna(1.0)
stock.to_csv(out / f"{code}.csv", index=False) # 保存新增字段
```

小练习：从日频宽表计算价格与收益率

使用作业答案生成的 wide：行索引为交易日，列索引第一层为 open, close, amount, volume, 第二层为股票代码。

- ① 使用成交额和成交量计算每只股票的日内成交均价。
- ② 使用开盘价和收盘价，计算每只股票的日内简单收益率序列。

小练习：从日频宽表计算价格与收益率

使用作业答案生成的 wide：行索引为交易日，列索引第一层为 open, close, amount, volume, 第二层为股票代码。

- ① 使用成交额和成交量计算每只股票的日内成交均价。
- ② 使用开盘价和收盘价，计算每只股票的日内简单收益率序列。

$$\bar{P}_{i,t} = \frac{\text{amount}_{i,t}}{\text{volume}_{i,t}},$$

$$r_{i,t}^{\text{intra}} = \frac{\text{close}_{i,t}}{\text{open}_{i,t}} - 1.$$

Python 参考答案

```
avg_price = wide["amount"] / wide["volume"] # 日内成交均价宽表
intraday_ret = wide["close"] / wide["open"] - 1 # 日内收益率宽表
```

思考：上述两个字段是否受到复权因子影响？

从单期收益到净值

若初始净值 $NAV_0 = 1$, 简单收益率为 R_t , 则

$$NAV_t = NAV_{t-1}(1 + R_t) = \prod_{s=1}^t (1 + R_s).$$

- 累计收益率:

$$CR_t = NAV_t - 1.$$

- 净值曲线比单期收益率更直观, 但更容易掩盖阶段性风险。

定义

历史最高净值:

$$H_t = \max_{s \leq t} NAV_s.$$

回撤:

$$DD_t = \frac{NAV_t}{H_t} - 1.$$

最大回撤:

$$MDD = \min_t DD_t.$$

- 回撤描述的是“从历史高点亏了多少”，比波动率更贴近投资体验。

案例：沪深 300 Buy-and-Hold 的净值与回撤



收益率统计特征

指标	含义	常见用途
均值	平均单期收益	收益水平
波动率	收益率标准差	风险水平
偏度	分布是否左右不对称	极端收益方向
超额峰度	相对正态分布的尾部厚度（正态为 0）	极端风险
最大回撤	净值相对历史高点最大跌幅	路径风险
Sharpe	单位波动率对应的超额收益	风险调整收益

Sharpe 越高，表示单位波动风险获得的超额收益越多，但它不能反映偏度、尾部损失和回撤路径。

夏普率 (Sharpe Ratio): 频率和样本周期都会改变结果

令 $R_t - R_{f,t}$ 为单期超额收益, 年化 Sharpe 为

$$\text{Sharpe}_{\text{ann}} = \frac{\overline{R_t - R_{f,t}}}{s(R_t - R_{f,t})} \sqrt{K},$$

其中 K 是一年内的期数: 日频通常取 252, 周频取 52, 月频取 12。

收益率频率不同

- 日、周、月收益率的均值和波动率估计不同。
- 用 $\sqrt{K}$ 年化隐含收益率独立同分布等假设。
- 存在自相关、异方差或非线性收益时, 各频率结果不会完全一致。

样本区间不同

- 近 1 年、近 3 年与近 5 年包含的市场状态不同。
- 牛市、熊市和震荡期会改变平均超额收益与波动率。
- 样本越短, Sharpe 的估计通常越不稳定。

比较前必须报告计算口径

样本起止日期、收益率频率、年化系数和无风险利率必须同时说明。

Sharpe 是依赖样本和计算口径的统计估计, 不是策略的固定属性。

比较资产或策略：绩效口径必须一致

绩效指标的数值依赖样本和计算规则；只有先统一口径，比较结果才有意义。

1. 比较区间一致

- 使用相同的起止日期和共同交易日。
- 牛市样本与熊市样本不能直接比较。

2. 收益口径一致

- 原始价、复权价与全收益口径不能混用。
- 策略收益应同时采用税费前或税费后口径。

3. 频率与年化方法一致

- 日频、周频、月频的年化系数分别约为 252、52、12。
- $\sigma_{\text{ann}} = \sigma_{\text{period}} \sqrt{K}$, $S_{\text{ann}} = S_{\text{period}} \sqrt{K}$ 。

4. 基准与假设一致

- 使用相同的无风险利率和市场基准。
- 统一交易成本、再投资和缺失值处理方式。

例：A 的日频 Sharpe 为 0.06，B 的月频 Sharpe 为 0.25，二者不能直接比较。年化后

$$S_A = 0.06\sqrt{252} = 0.95, \quad S_B = 0.25\sqrt{12} = 0.87.$$

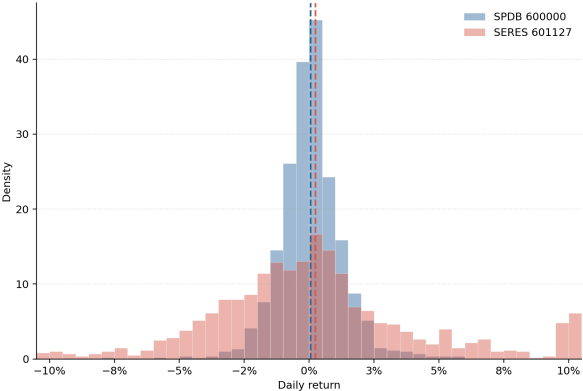
先统一样本与计算规则，再比较收益、风险和风险调整绩效。

案例：浦发银行与赛力斯的收益率特征

采用 2021–2025 年前复权日收盘价，计算简单收益率及其统计特征。

指标	浦发银行 600000	赛力斯 601127
日均收益率	0.046%	0.240%
年化波动率	19.5%	63.2%
偏度	0.15	0.53
超额峰度	3.87	0.66
最大回撤	-33.8%	-71.7%
年化 Sharpe	0.59	0.96

Sharpe 假设无风险利率为 0，按日均收益和日波动率年化。



Forward-adjusted close, 2021-2025; dashed lines denote means

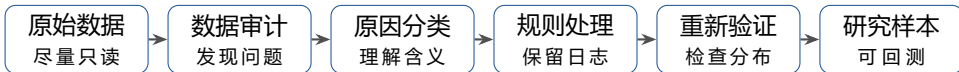
观察： 上述两个资产的收益率统计特征告诉了我们什么？

本讲位置

- 1 金融数据处理的基本流程
- 2 金融数据及其结构
- 3 价格口径、收益率与基础绩效
- 4 数据清洗与时间对齐**
- 5 课堂实践

数据清洗是在恢复真实的研究样本

清洗的目标：判断每条数据在当时代表什么。



需要区分

- 没有数据、没有交易与价格为零。
- 数据错误、公司行为与真实极端行情。

必须保留

- 原始文件、处理规则与问题日志。
- 每次处理前后的样本变化记录。

做人的底线：不能为了让策略结果更好看而修改样本。

第一步：建立数据质量报告

结构检查

- 日期范围、字段类型与排序。
- date + code 是否唯一。
- 每日股票数量是否异常突变。

数值检查

- 各字段缺失比例。
- OHLC 价格关系是否成立。
- 成交量、市值是否出现负数。

质量报告：数据准确性的最低要求。质量报告没有问题不代表数据完全可信；但如果质量报告发现问题，则数据完全不可信。

```
key = ["date", "code"] # 定义记录的唯一键
print(panel.dtypes) # 检查字段类型
print(panel.duplicated(key).sum()) # 统计重复记录
print(panel.isna().mean().sort_values()) # 计算缺失比例

bad_ohlc = panel[ # 找出价格关系异常的记录
    (panel["low"] > panel[["open", "close"]].min(axis=1))
    | (panel["high"] < panel[["open", "close"]].max(axis=1))
]

coverage = panel.groupby("date")["code"].nunique() # 每日股票数
```

缺失值不等于零：先判断经济含义

情况	价格记录	收益率处理	股票池与交易处理
节假日	无记录	不计算	不是交易日
停牌	旧价格或缺失	由估值价格得到 0, 或保留缺失	保留持仓, 但不能交易
尚未上市	缺失	缺失	不应进入当期股票池
已退市	终止记录	需处理退市收益	此后移出股票池
数据漏抓	异常缺失	暂不计算	回查数据源后再决定

可以前向填充价格

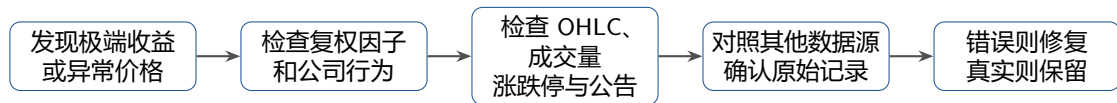
用于估值或保持持仓市值连续, 但必须同时保留停牌和可交易状态。

不要前向填充收益率

收益率是价格变化的结果; 直接填充会制造不存在的收益信息。

价格矩阵、收益率矩阵和可交易矩阵应分别维护。

异常值：先解释，再决定是否处理



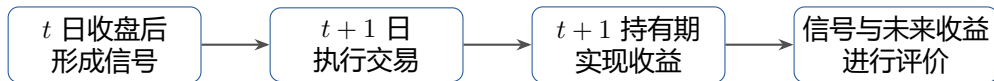
可能是数据错误

- 小数点错位、重复记录、复权两次。
- OHLC 关系矛盾或成交量为负。

也可能是真实风险

- 涨跌停、复牌补跌、重大事件冲击。
- 极端收益正是尾部风险的一部分。

时间对齐：信息何时真正可得？



错误对齐

$$\text{signal}_{i,t} \longrightarrow R_{i,t}$$

用收盘后才知道的信号，假设已经按当日收盘价成交。

正确对齐

$$\text{signal}_{i,t} \longrightarrow R_{i,t+1}$$

```
panel = panel.sort_values(["code",  
"date"])  
next_ret = groupby(code).ret.shift(-1)
```

- 财务数据按公告日进入信息集，不能按报表期末日期提前使用。
- 市值、股票池状态和交易状态也必须采用**当时可得**的历史记录。

对齐的不只是日期，还有历史股票池

日期对齐

- 先建立交易所标准交易日历。
- 再映射每只股票的上市、停牌和退市状态。
- 内连接会缩短样本；外连接必须解释缺失。

一行 = 同一个可比较时点

股票池对齐

- $\mathcal{U}_t$ 只能包含 t 时刻已上市且满足条件的股票。
- 需要保留后来退市、ST 和长期停牌的股票。
- 使用今天仍上市的股票回测过去，会产生幸存者偏差。

$$\mathcal{U}_t \neq \mathcal{U}_T$$

清洗完成后：重新验证研究样本

结构验收

- ① 主键唯一，日期和代码排序正确。
- ② 股票数量变化能够由上市、退市解释。
- ③ 原始数据与处理后数据分别保存。
- ④ 每条清洗规则及影响行数均有日志。

统计验收

- ① 重新计算缺失率和异常记录数量。
- ② 检查收益率分布和极端收益日期。
- ③ 检查是否存在未来信息或错误填充。
- ④ 抽查若干股票并与原始行情核对。

最终对象	核心字段	后续用途
研究面板	date, code, market_cap, next_ret	排序与收益评价
状态字段	is_st, listed_days, tradable, amount	可实现性过滤
处理日志	规则、日期、股票与影响行数	复现与审计

最终目标：当时真实可得、可交易、可复现的研究样本。

判断题：以下数据处理是否正确？

请判断下列操作是否合理，并简要说明可能引入的偏差或风险。

- ❶ 研究 2015–2025 年的沪深 300 股票时，只保留在整个样本期内数据完整的 170 只股票，并将其作为固定资产池。
- ❷ 导入 2015–2025 年的全部股票（包括期间退市的股票），并在每个交易日根据当日 ST 状态，将 ST 股票剔除出当日资产池。
- ❸ 为构造 2020–2025 年的完整面板，将一只 2022 年才上市股票的首日开盘价向前填充至 2020 年。
- ❹ 鉴于 A 股存在涨跌停限制，将所有日涨跌幅绝对值超过 20% 的记录统一视为异常值，并截断至 $\pm 20\%$ 。
- ❺ 在计算股票历史收益率时，统一使用前复权价格。

判断题（续）：以下数据处理是否正确？

请判断下列操作是否合理，并简要说明可能引入的偏差或风险。

- ① 使用 2025 年末的沪深 300 成分股名单，回测 2015–2025 年的投资策略。
- ② 只要某行任意字段存在缺失值，就删除整行记录，再进行因子研究。
- ③ 将季度财务数据归入对应季度末，并从季度末次日起用于构造交易信号。
- ④ 使用当前的行业分类，对过去十年的股票数据进行行业中性化。
- ⑤ 发现除权除息日的收益率异常后，直接删除这些交易日的记录。
- ⑥ 将月度宏观数据按照统计月份与股票收益直接合并。

本讲位置

- 1 金融数据处理的基本流程
- 2 金融数据及其结构
- 3 价格口径、收益率与基础绩效
- 4 数据清洗与时间对齐
- 5 课堂实践

课堂实践一：比较两家上市公司的收益特征

自行选择两家上市公司，使用其历史行情比较收益与风险特征。

计算任务

- ① 下载两只股票的前复权日收盘价。
- ② 计算日简单收益率与累计净值。
- ③ 输出日均收益率、年化波动率、偏度、超额峰度、最大回撤和年化 Sharpe。
- ④ 分别绘制两只股票的日收益率直方图。

比较要求

- ① 使用相同的样本起止日期和共同交易日。
- ② 使用相同的复权方式、无风险利率和年化系数。
- ③ 直方图采用相同的分组区间和横坐标范围。
- ④ 用三句话解释两家公司收益分布和风险的主要差异。

提交内容

两家公司及选择理由、一张统计量对比表、收益率直方图，以及对比较结果的简要解释。

课堂实践二：持有沪市最小市值的 100 只股票

朴素策略

每个月末在沪市历史股票池中按照总市值从小到大排序，选择最小的 100 只股票，下一月等权持有，并与沪深 300 比较。

$$\mathcal{P}_t = \text{Bottom}100_{i \in \mathcal{U}_t^{\text{SH}}}(\text{MarketCap}_{i,t}), \quad R_{p,t+1} = \frac{1}{100} \sum_{i \in \mathcal{P}_t} R_{i,t+1}.$$

- 排序信息来自月末 t ，组合收益必须来自下一持有期 $t+1$ 。
- 先不考虑交易成本和可交易性过滤，观察策略的纸面效果。

注意：

要判断策略是否使用了真实历史股票池、可得信息和可成交价格。

上节课作业数据：

每行表示一只沪市股票在一个月末的状态。

样本日期为 2015 年 1 月至 2025 年 9 月；最后一个月没有完整的下一期收益，回测会自动剔除。

字段	含义
date, code	形成组合的月末日期与股票代码
market_cap	当时可得的总市值
next_ret	从本期形成日到下一期的股票收益
benchmark_ret	同一持有期的基准收益
is_st, listed_days	ST 状态与上市天数
tradable, amount	可交易状态与成交额

构造朴素沪市小市值组合

```
panel = pd.read_csv("data/raw/sh_stock_monthly_panel.csv") # 读取月度面板
panel["date"] = pd.to_datetime(panel["date"]) # 转换日期类型

# 只保留代码以 6 开头的沪市股票
panel = panel[panel["code"].str.startswith("sh.6")].copy() # 筛选沪市股票

panel["size_rank"] = panel.groupby("date")["market_cap"].rank(
    method="first", ascending=True
) # 市值由小到大排名
selected = panel[panel["size_rank"] <= 100].copy() # 选前100只

strategy_ret = selected.groupby("date")["next_ret"].mean() # 等权收益
benchmark_ret = panel.groupby("date")["benchmark_ret"].first() # 基准收益

result = pd.concat([
    strategy_ret.rename("Shanghai Smallest 100"),
    benchmark_ret.rename("CSI 300")
], axis=1).dropna() # 按日期对齐并删除缺失期
```

检查：每个形成日应选出 100 只股票，并核对 100 只股票是否都有下一期收益；mean() 会自动跳过缺失值。

观察策略效果

```
nav = (1 + result).cumprod() # 将收益率转换为净值
nav.plot(figsize=(10, 5), logy=True) # 使用对数纵轴
plt.ylabel("Net value (log scale)")
plt.title("Shanghai Smallest 100 vs CSI 300")
plt.tight_layout()
plt.show()

summary = pd.DataFrame({
    "annual_return": (1 + result).prod() ** (12 / len(result)) - 1, # 年化收益
    "annual_vol": result.std() * 12 ** 0.5, # 年化波动率
    "sharpe": result.mean() / result.std() * 12 ** 0.5, # 年化Sharpe
    "max_drawdown": (nav / nav.cummax() - 1).min() # 最大回撤
})
print(summary) # 输出绩效表
```

练习效果：沪市最小市值 100 只股票



月度等权纸面回溯；未计交易成本与成交限制

纸面收益显著高于基准，这个收益真的能够被投资者获得吗？

这个策略可以实现吗？

- **幸存者与退市偏差**：是否包含后来退市的公司？缺失的退市收益是否被 `mean()` 静默跳过？
- **不可交易**：停牌、涨停可能买不到，跌停可能卖不出。
- **流动性与容量**：小市值股票成交额低，等权交易可能显著冲击价格。
- **交易成本**：频繁更换最小 100 只股票会产生佣金、印花税和滑点。
- **风险标签**：ST、退市风险和上市初期股票可能集中进入组合。
- **信息时点**：市值、股本和复权数据是否在形成组合时真实可得？

历史收益很高不代表组合可成交、可容纳资金或可执行。

加入基本可实现性过滤

```
min_amount = 10_000 # 1,000万元; amount 的单位为千元

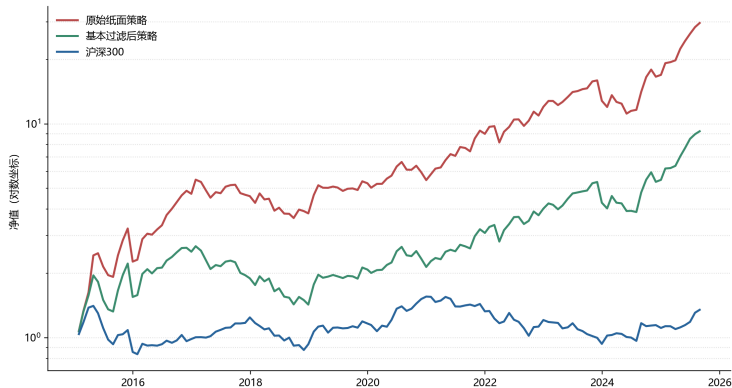
eligible = panel[
    (panel["is_st"] == 0)
    & (panel["listed_days"] >= 120)
    & (panel["tradable"] == 1)
    & (panel["amount"] >= min_amount)
].copy() # 应用基本可交易性过滤

eligible["size_rank"] = eligible.groupby("date")["market_cap"].rank(
    method="first", ascending=True
) # 在可选股票中重新排名
selected_realistic = eligible[eligible["size_rank"] <= 100] # 选前100只

filtered_ret = selected_realistic.groupby("date")["next_ret"].mean() # 等权收益
filtered_result = pd.concat([
    filtered_ret.rename("Filtered Smallest 100"),
    benchmark_ret.rename("CSI 300")
], axis=1).dropna() # 与基准按日期对齐
```

- 重新计算净值，并比较过滤前后的收益、波动率和最大回撤。
- 过滤条件包括剔除 ST、上市不足 120 天、不可交易以及日成交额低于 1,000 万元的股票。

过滤后结果：收益下降，但已基本具备可行性



组合	期末净值	年化收益	年化波动	年化 Sharpe	最大回撤
原始纸面策略	29.67	37.4%	33.4%	1.12	-33.8%
基本过滤后策略	9.24	23.2%	31.5%	0.82	-46.7%
沪深 300	1.35	2.9%	20.6%	0.24	-40.6%

过滤降低了纸面收益，但没有自动解决交易成本、冲击成本和策略容量。

- ① 沪市最小市值 100 只股票策略与沪深 300 的对数净值曲线。
- ② 原始策略与可实现性过滤后策略的基础绩效比较。
- ③ 任选一个调仓月，列出进入和退出组合的股票及其交易状态。
- ④ 简要回答：这个沪市小市值策略是否能够真实实现？为什么？
- ⑤ 提出你的改进方案。

本讲小结

- 金融数据处理的核心是统一价格口径、时间口径和样本口径。
- 简单收益率适合组合加权，对数收益率适合时间加总和部分统计建模。
- 净值曲线来自收益率累乘，回撤来自净值相对历史高点的跌幅。
- 缺失、异常、停牌、复权和时间错位都会污染后续因子检验与回测。
- 沪市最小市值 100 只股票的纸面收益不等于可实现收益，还需处理历史股票池、交易限制、流动性、成本和容量。